PROJECT DOCUMENTATION

Azure Highly Available Web Application

Deployment with Virtual Machine Scale Sets, Standard Load Balancer, NAT Gateway & CPU-Based Auto-Scaling

Prepared by: Parikshit Arer
Resource Group: Project-1
Azure Region: East US 2
Build Window: October 1 – 3, 2026

Table of Contents

This document records the end-to-end build of the Project-1 Azure environment, in the order the resources were created, with a screenshot, a configuration snapshot, and an explanation for every step.

1. Project Overview

2. Solution Architecture

3. Phase 1 — Foundation Setup

Steps 1.1, 1.2, 1.3, 1.4

4. Phase 2 — Compute Deployment: VM Scale Set & Web Server Automation

Steps 2.1, 2.2, 2.3, 2.4

5. Phase 3 — Outbound Connectivity: NAT Gateway

Steps 3.1, 3.2

6. Phase 4 — Load Balancing

Steps 4.1, 4.2, 4.3, 4.4

7. Phase 5 — Validation, Scaling & End-to-End Test

Steps 5.1, 5.2, 5.3, 5.4

8. Final Review & Resource Inventory

9. Recommendations & Next Steps

10. Conclusion

Appendix A — Supplementary Autoscale Exercise

1. Project Overview

Design, deploy, and validate a highly available web-application architecture on Microsoft Azure that can absorb both instance failure and traffic spikes without manual intervention. The web tier runs on a Virtual Machine Scale Set distributed across two availability zones, is fronted by a Standard public Load Balancer, is reachable only through an explicit NSG allow-list, and scales automatically between 2 and 3 instances based on CPU utilization — all provisioned through infrastructure automation (ARM deployment + a Custom Script Extension) rather than manual, point-and-click server configuration.

Resource Group	Project-1	**Azure Region**	East US 2
Build Window	October 1 – 3, 2026	**Prepared by**	Parikshit Arer

Technologies & Azure Services Used

Azure Resource Groups	Logical container for every resource in the project
Azure Virtual Network (VNet)	Private address space, subnetted into three /24 ranges
Network Security Groups (NSG)	Subnet-level and NIC-level inbound traffic filtering
Virtual Machine Scale Set (VMSS)	Ubuntu 24.04 LTS instances, zone-distributed, auto-scaling
Custom Script Extension	Automated, repeatable nginx installation and configuration
Azure NAT Gateway	Centralized, predictable outbound internet connectivity
Azure Load Balancer (Standard)	Public traffic distribution and backend health probing
Azure Monitor Autoscale	CPU-threshold-based automatic instance scaling
Azure Serial Console	Agentless, in-portal VM diagnostics without SSH

2. Solution Architecture

Two views of the same deployment: the request/traffic flow from the internet down to an individual VM, and the network layout showing how the virtual network's three subnets are used.

Traffic Flow

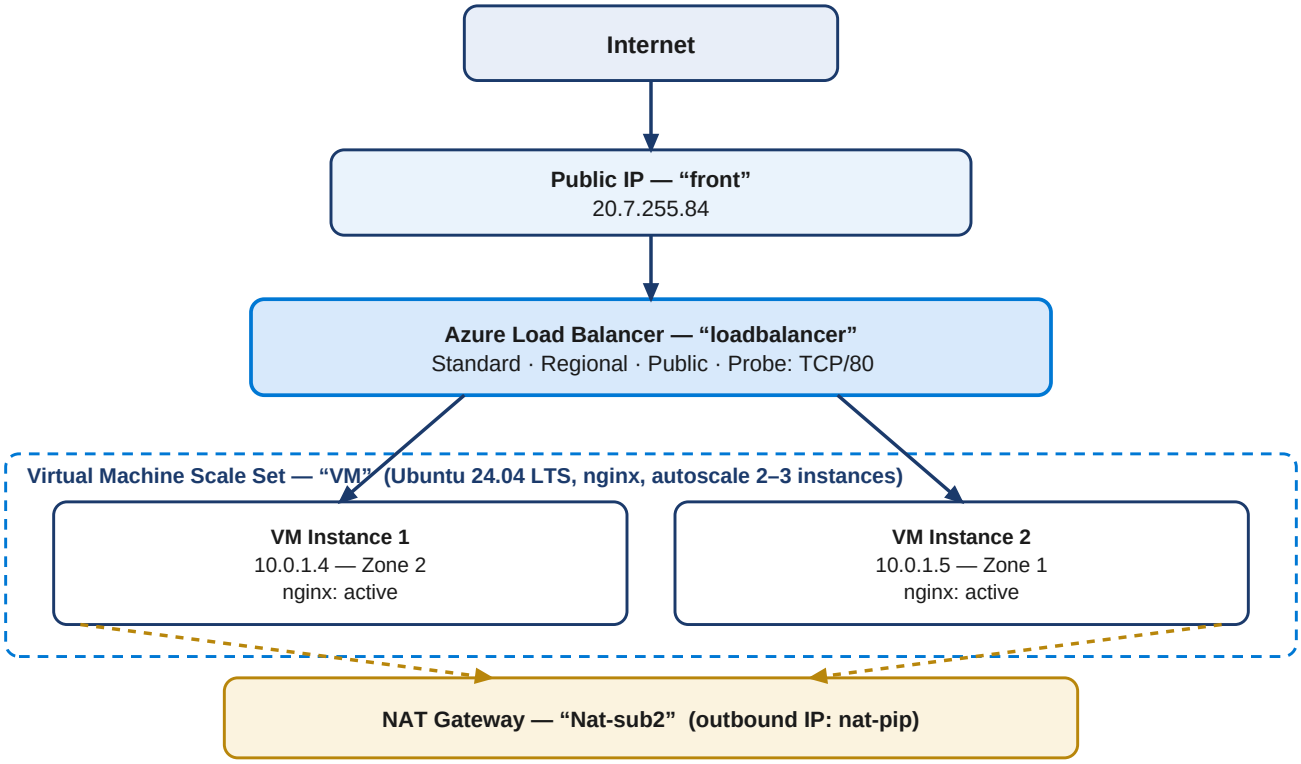


Figure 2.1 — Inbound requests travel Internet → Public IP → Load Balancer → Backend Pool. Outbound traffic from the instances (package installs, updates) takes a separate, NAT-Gateway-only path back out to the internet.

Network Layout

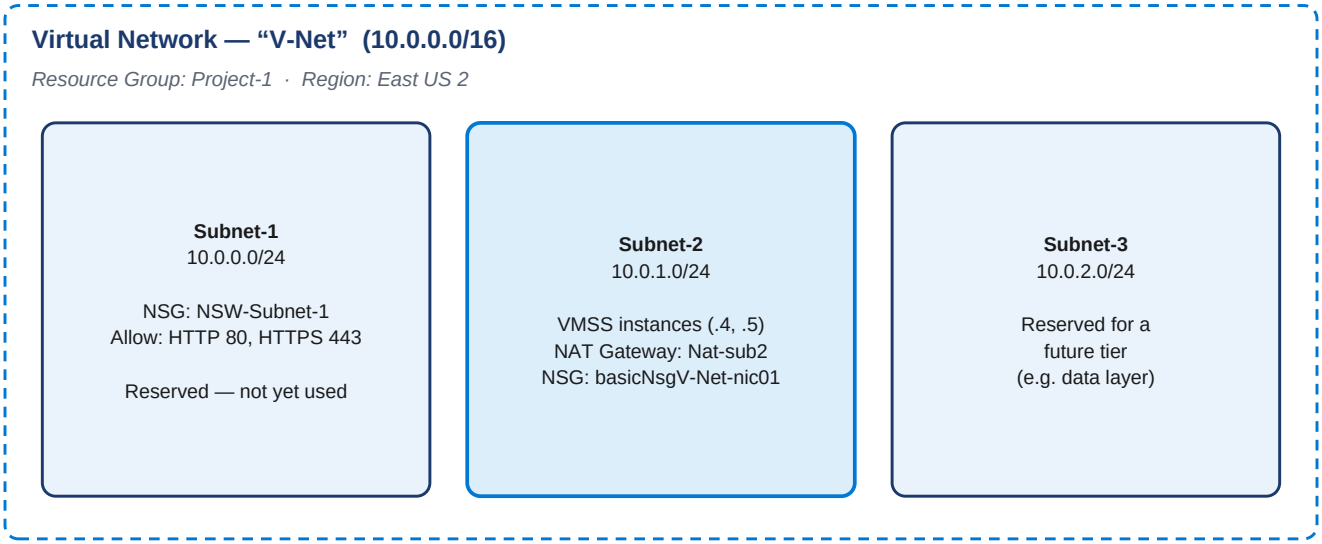


Figure 2.2 — The /16 virtual network is split into three /24 subnets. Only Subnet-2 is actively in use by this build; Subnet-1 and Subnet-3 are provisioned and reserved for future tiers.

3. Phase 1 — Foundation Setup

PHASE 1

Foundation Setup

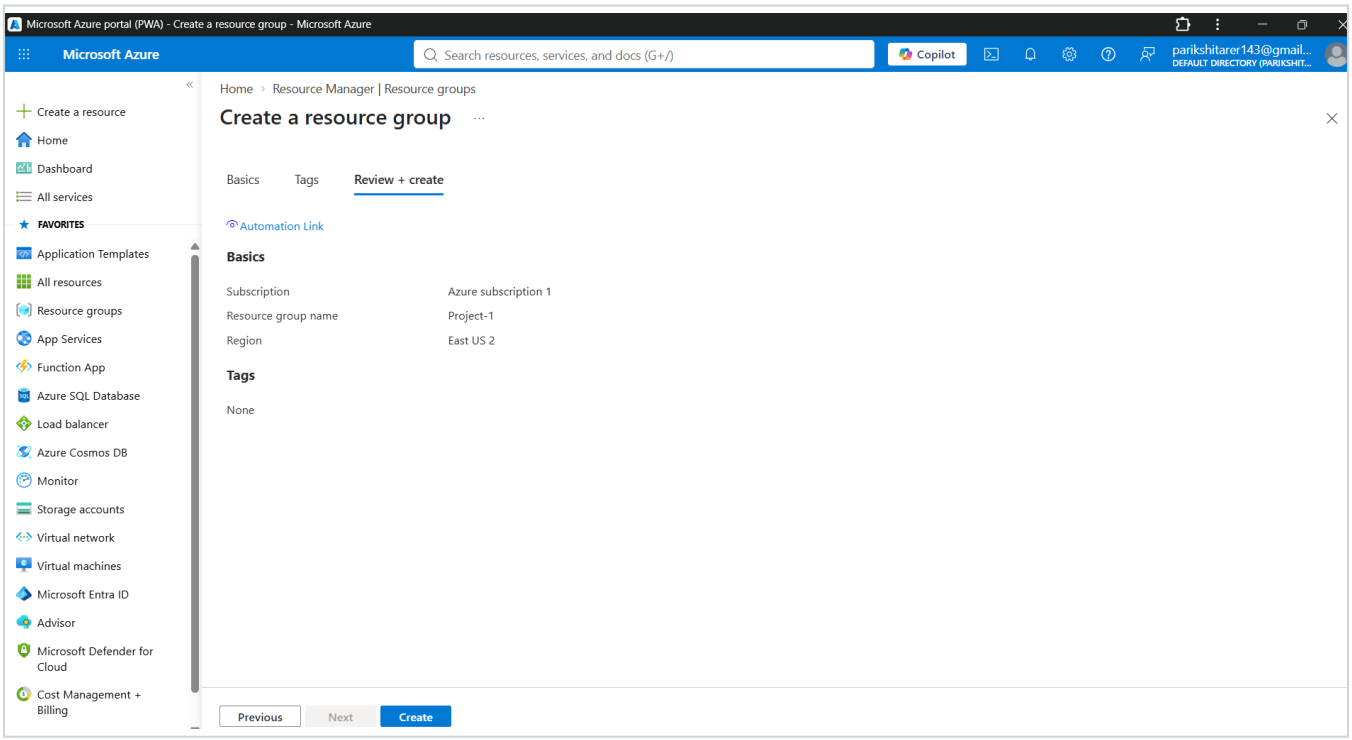
Establish the resource group, the virtual network, its subnets, and the first layer of network security before any compute is deployed.

1.1 Create the Resource Group

Resource group	Project-1
Subscription	Azure subscription 1
Region	East US 2

Every resource in this project — network, compute, load balancer, and NAT gateway — is provisioned inside a single resource group named **Project-1** in **East US 2**.

Grouping everything together keeps the architecture easy to manage, tag, cost-track, and tear down as one unit, and it's the first resource created so every later step has somewhere to land.



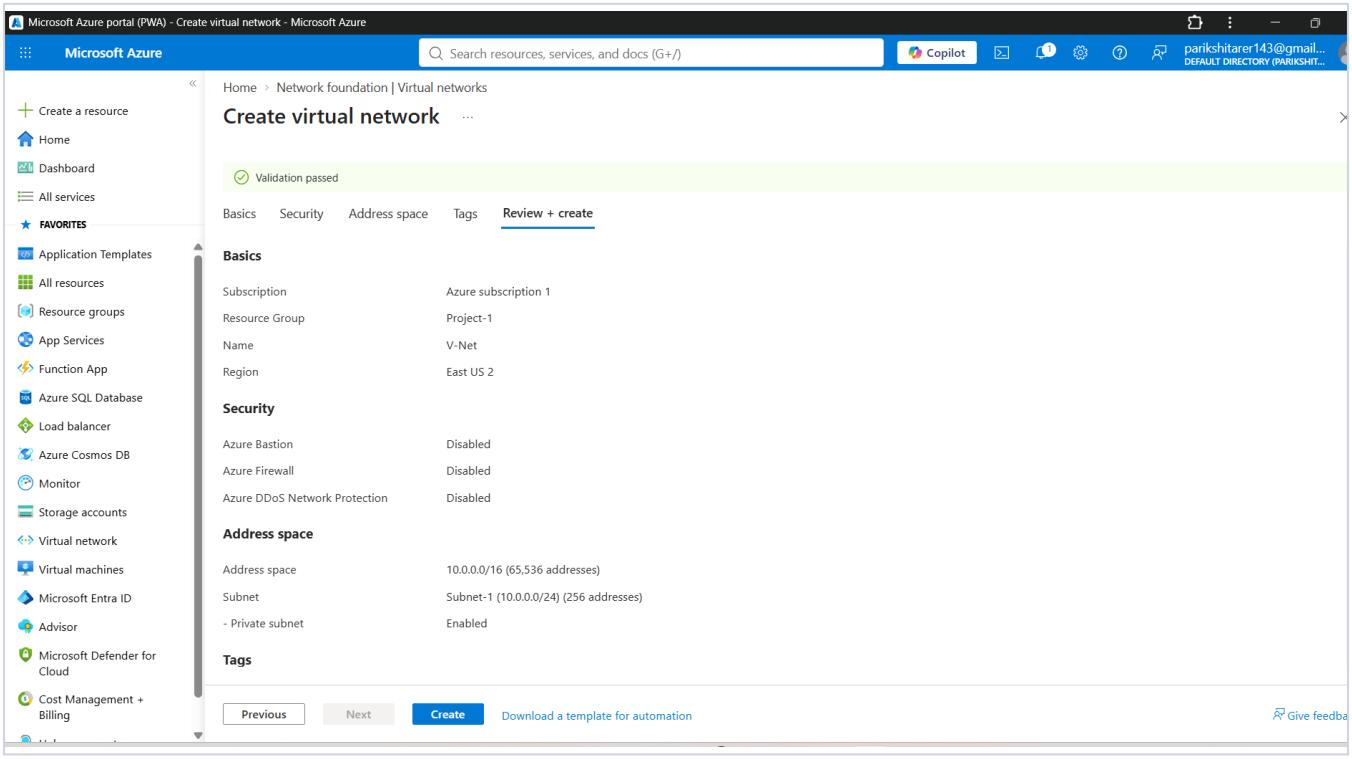
Resource group “Project-1” — Review + create

1.2 Create the Virtual Network

Name	V-Net
Address space	10.0.0.0/16 (65,536 addresses)
Initial subnet	Subnet-1 — 10.0.0.0/24 (256 addresses)
Private subnet	Enabled
Bastion / Firewall / DDoS Protection	Disabled

The virtual network **V-Net** is created with a /16 address space — far more room than this project needs, but enough headroom to carve out additional subnets later without re-planning the network.

Azure Bastion, Azure Firewall, and Azure DDoS Network Protection are deliberately left disabled at this stage to keep the build lightweight; perimeter security here is handled instead with Network Security Groups (Step 1.4).



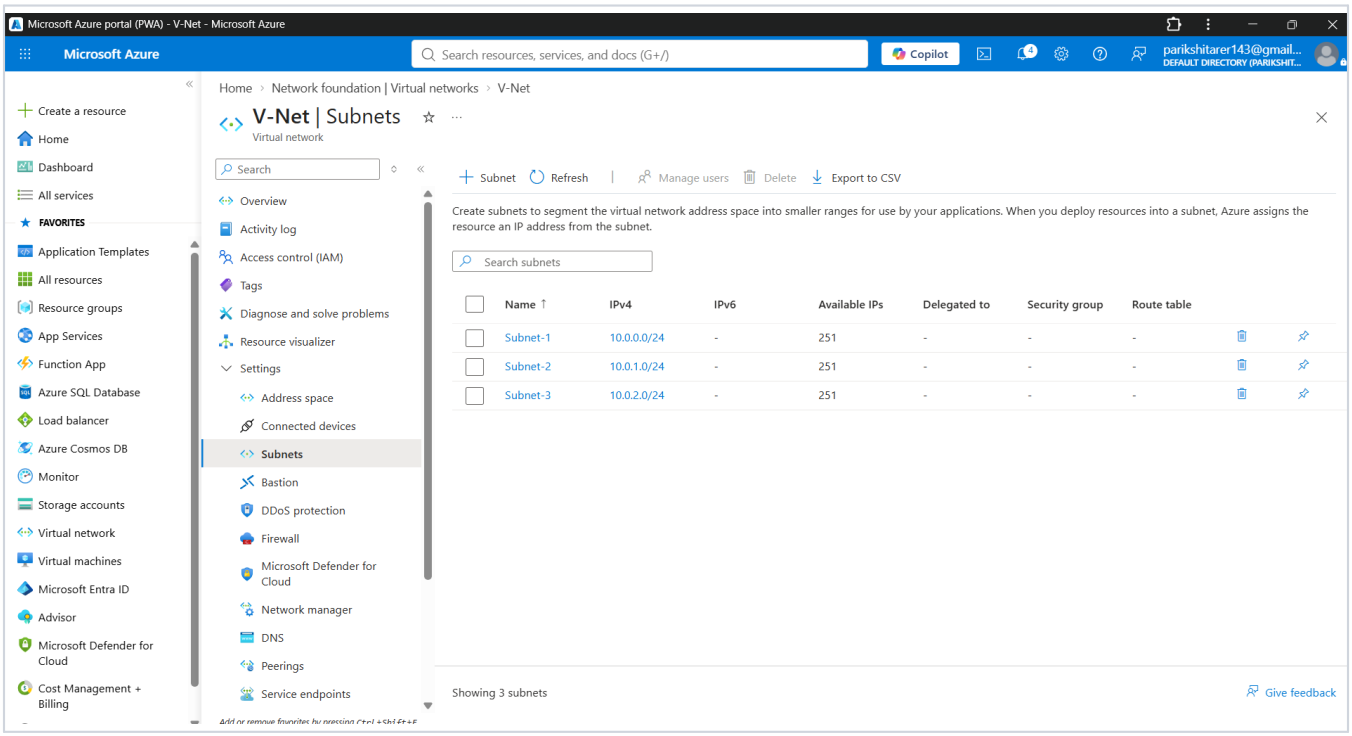
V-Net — Review + create, showing address space and security defaults

1.3 Provision the Subnets

Subnet-1	10.0.0.0/24 — 251 usable IPs
Subnet-2	10.0.1.0/24 — 251 usable IPs
Subnet-3	10.0.2.0/24 — 251 usable IPs

Three /24 subnets are carved out of the V-Net address space. **Subnet-2** is the one that ends up hosting the application tier — the VM Scale Set instances and the NAT Gateway, confirmed later in Phases 2 and 3.

Subnet-1 and **Subnet-3** are provisioned but reserved for future use (for example, a dedicated gateway subnet and a future data tier), keeping the network ready to grow without re-addressing anything already deployed.



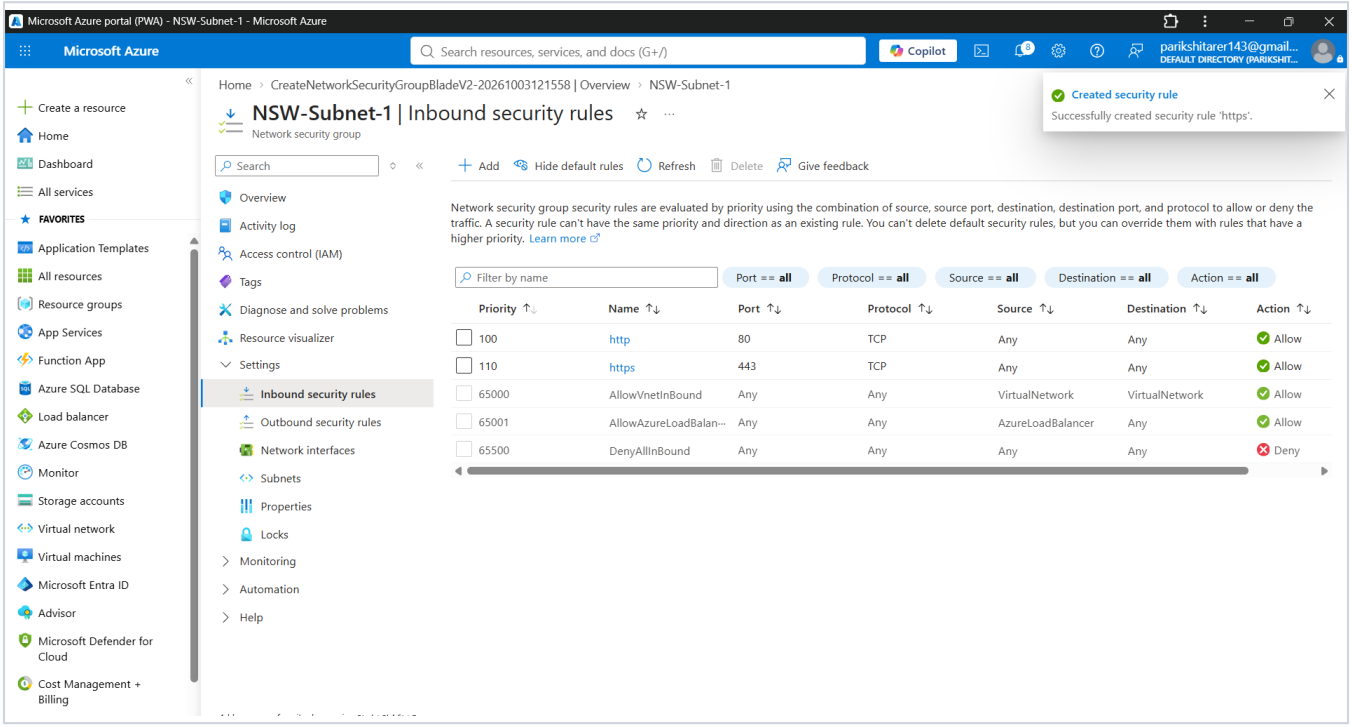
V-Net | Subnets — three /24 subnets provisioned

1.4 Configure the Network Security Group

NSG name	NSW-Subnet-1
Rule 100	http — TCP 80 — Any → Any — Allow
Rule 110	https — TCP 443 — Any → Any — Allow
Defaults (65000–65500)	AllowVnetInBound, AllowAzureLoadBalancerInBound, DenyAllInBound

Two custom inbound rules are added on top of Azure's three non-removable default rules: one allowing **TCP 80 (HTTP)** and one allowing **TCP 443 (HTTPS)**, both from any source.

Everything else stays blocked by the default **DenyAllInBound** rule at priority 65500 — an explicit, minimal allow-list rather than a broad opening, and the same layered-NSG pattern this project later repeats at the NIC level in Step 2.1.



NSW-Subnet-1 | Inbound security rules — HTTP and HTTPS allowed

4. Phase 2 — Compute Deployment: VM Scale Set & Web Server Automation

PHASE 2

Compute Deployment: VM Scale Set & Web Server Automation

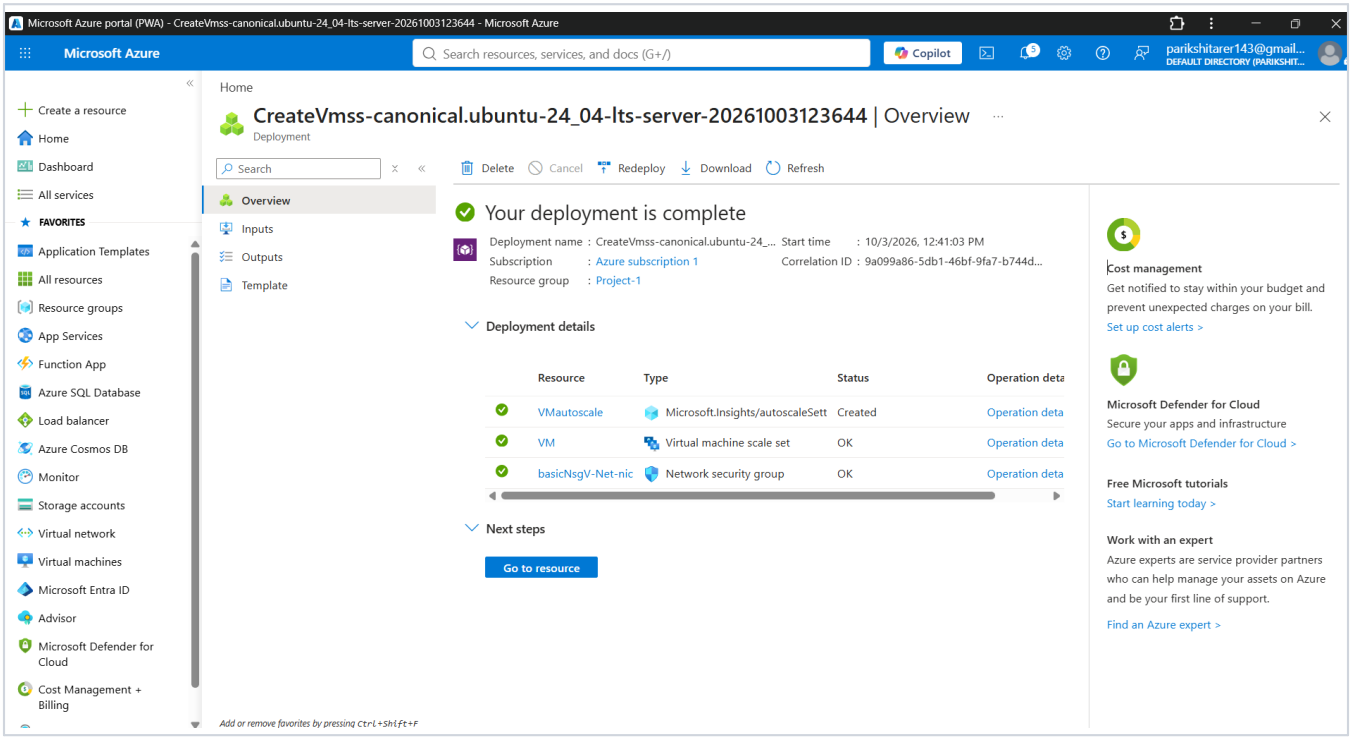
Deploy the Ubuntu scale set and bring every instance to a known-good, nginx-serving state using a single repeatable script instead of manual configuration.

2.1 Deploy the Virtual Machine Scale Set

Deployment	CreateVmss-canonical.ubuntu-24_04-lts-server
Image	Canonical — Ubuntu Server 24.04 LTS
Resource group	Project-1
Resources created	VMautoscale (autoscale settings), VM (scale set), basicNsgV-Net-nic (NSG)

A single ARM deployment provisions three resources together: the scale set itself (**VM**), a default autoscale-settings object (**VMautoscale**) that Phase 5 later tunes, and a second, **NIC-level** network security group (**basicNsgV-Net-nic**) that Azure generates automatically when “Basic” NIC networking is chosen in the scale-set wizard.

This gives the deployment two independent layers of NSG filtering — one at the subnet (NSW-Subnet-1, Step 1.4) and one at the NIC — reinforcing the HTTP/HTTPS-only allow-list from both directions.



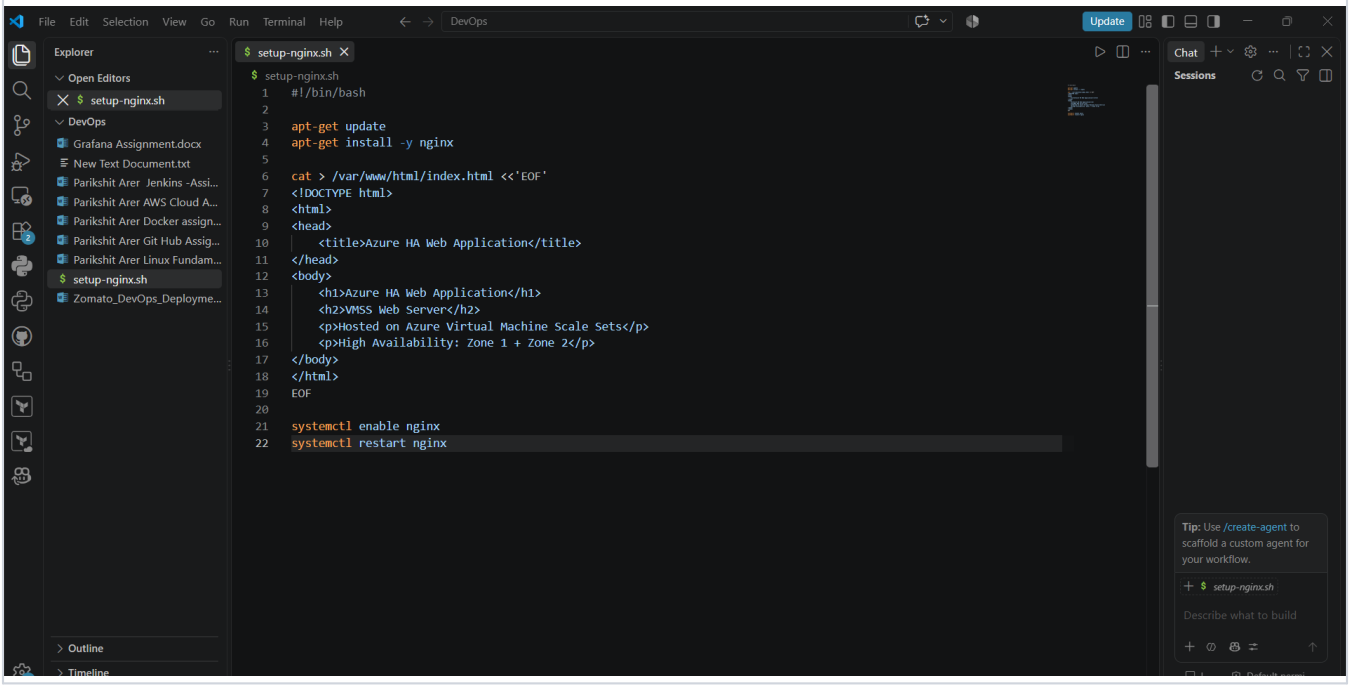
VMSS deployment — Your deployment is complete

2.2 Author the Web-Server Bootstrap Script

File	setup-nginx.sh
Installs	nginx (via apt-get)
Writes	/var/www/html/index.html — custom landing page
Finishes with	systemctl enable nginx; systemctl restart nginx

Rather than logging into each instance by hand, the web-server setup is captured as a single, idempotent Bash script: update the package index, install nginx, overwrite the default landing page with a custom “Azure HA Web Application” page (labeled “VMSS Web Server” and noting “High Availability: Zone 1 + Zone 2”), then enable and restart the nginx service so it survives a reboot.

This script is exactly what the Custom Script Extension runs on every instance in the scale set, guaranteeing identical configuration across all of them — no server is configured differently from any other.



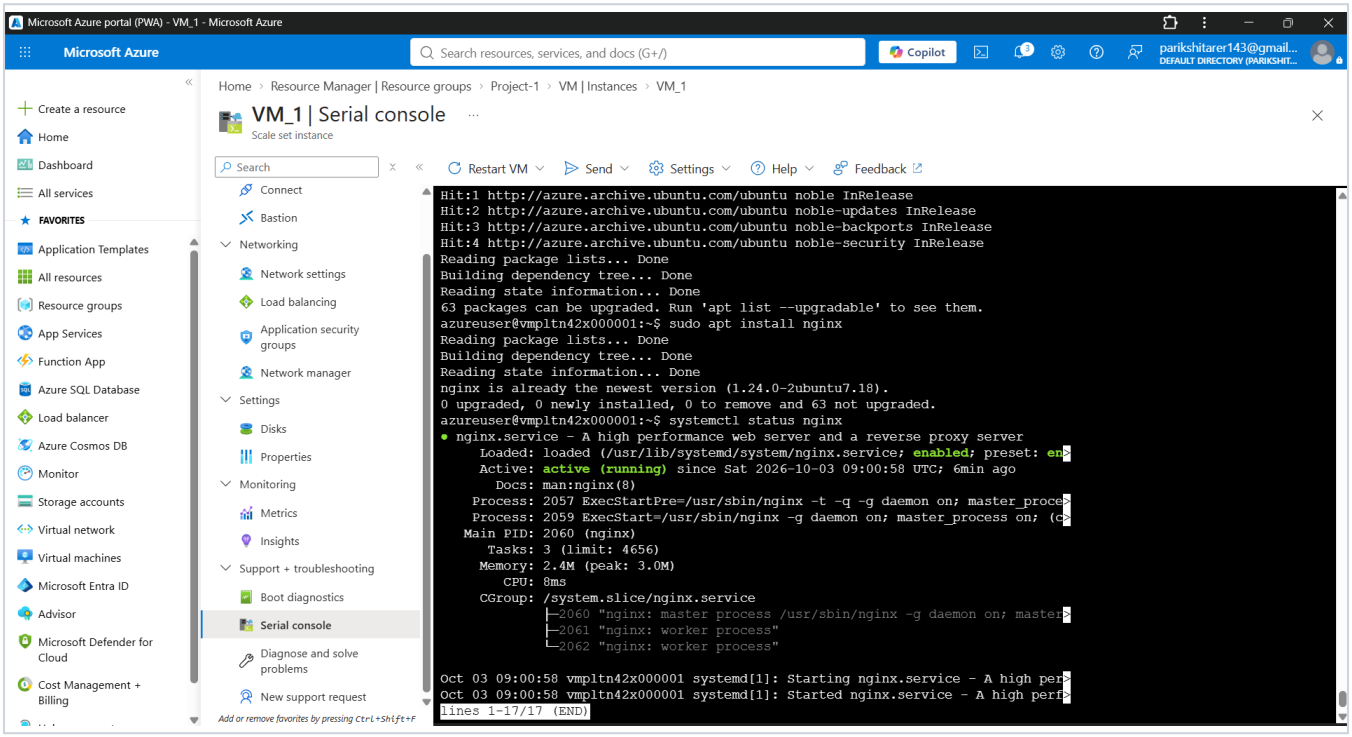
setup-nginx.sh in VS Code — the Custom Script Extension payload

2.3 Verify the Web Server — First Check

Instance	VM_1
Service	nginx.service
State	active (running)
Started	2026-10-03 09:00:58 UTC

Using the **Azure Serial Console** — a direct, portal-based console session that works even without SSH, a public IP, or an open management port — systemctl status nginx is run against the first scale-set instance.

The output confirms nginx is **enabled** and **active (running)**, validating that the Custom Script Extension executed successfully on first boot, with no manual intervention required.



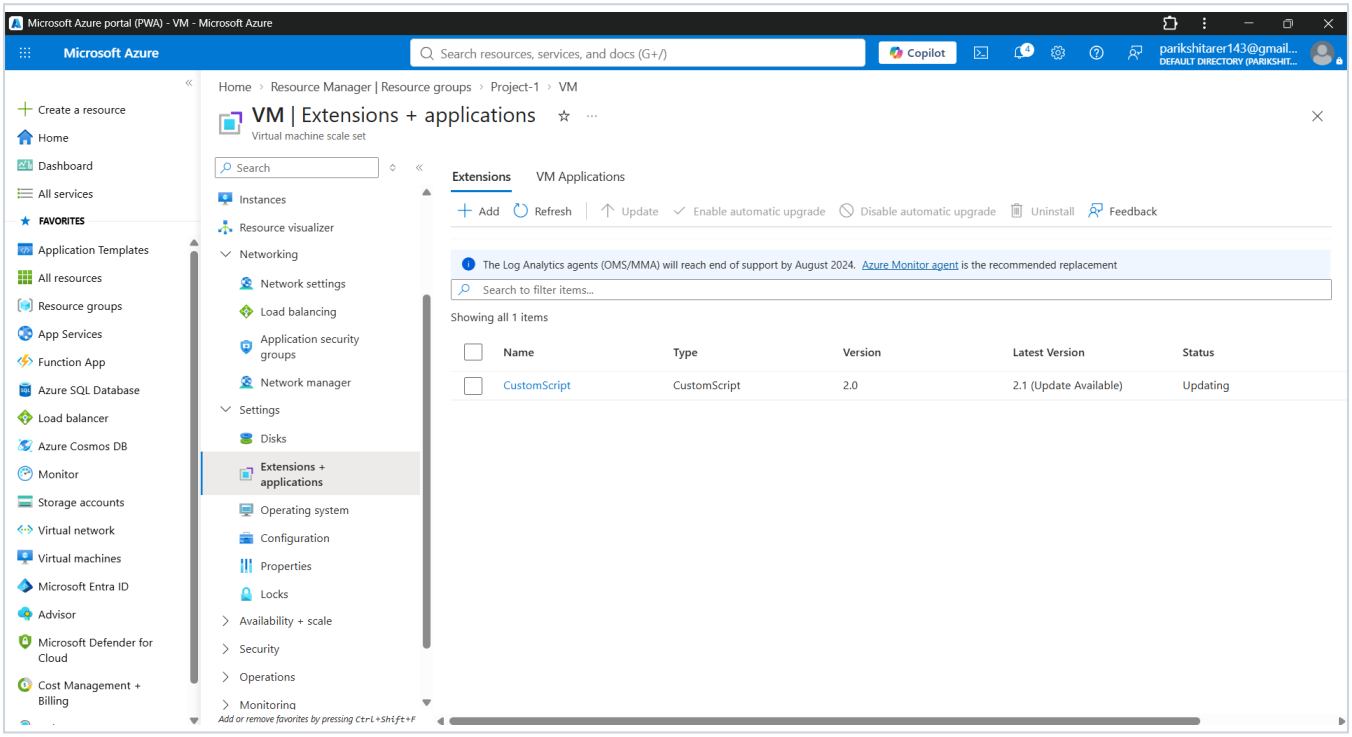
VM_1 | Serial console — systemctl status nginx

2.4 Monitor the Extension Rollout

Extension	CustomScript
Installed version	2.0
Latest available	2.1 (Update Available)
Status	Updating

The **Extensions + applications** blade shows the CustomScript extension mid-update, with a newer minor version (2.1) available from the publisher.

Tracking extension status here is a quick way to confirm that **automation** — not manual configuration — is what brings every instance in the scale set to a known-good state, and it foreshadows the second serial-console check in Phase 5.



VM | Extensions + applications — CustomScript mid-update

5. Phase 3 — Outbound Connectivity: NAT Gateway

PHASE 3

Outbound Connectivity: NAT Gateway

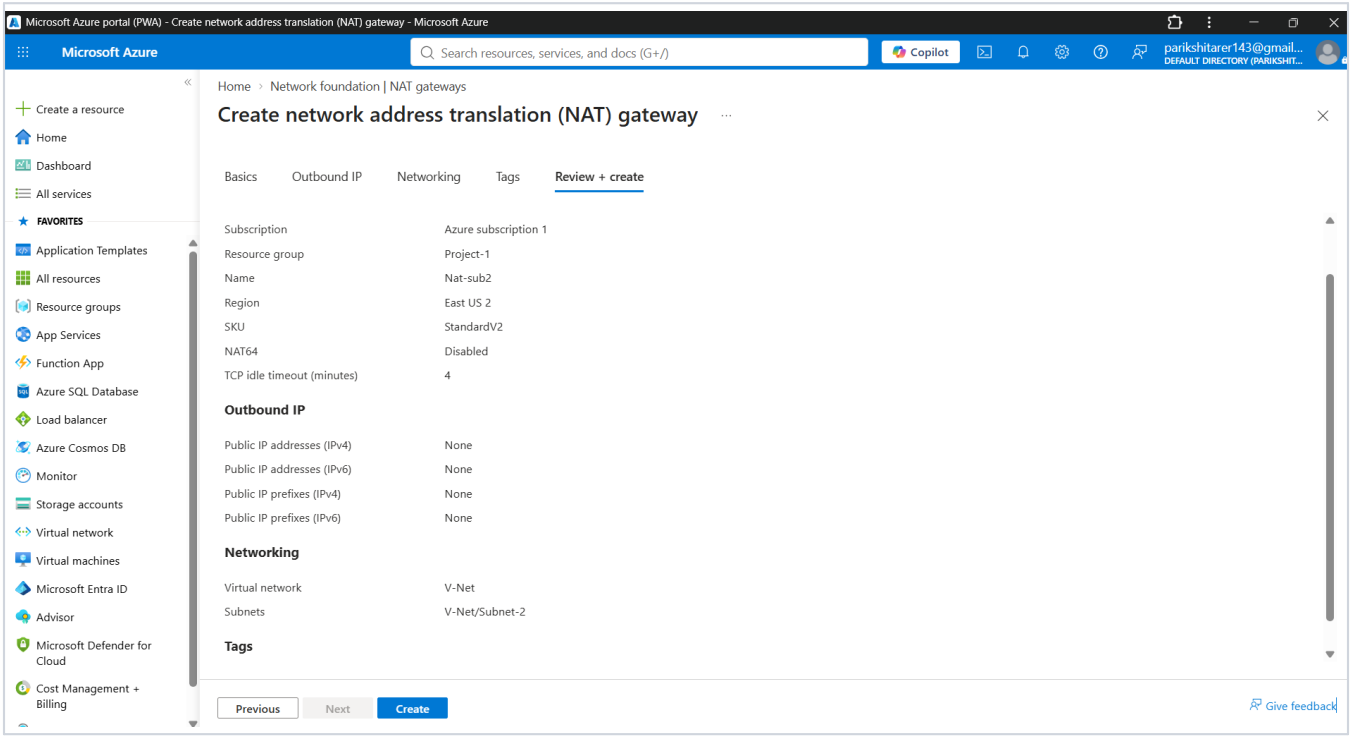
Give the private-subnet instances a single, predictable way to reach the internet for updates and package installs, without assigning any of them a public IP.

3.1 Create the NAT Gateway

Name	Nat-sub2
SKU	Standard V2
Region	East US 2
NAT64	Disabled
TCP idle timeout	4 minutes
Outbound public IP at creation	None selected

Because the application-tier VMs carry no public IPs on their NICs, outbound internet access — needed for apt-get update, package installs, and OS patching — is provided centrally through a **NAT Gateway** instead of giving every instance its own public IP.

That means a smaller attack surface and one single, predictable outbound address for the whole subnet, rather than many independently-managed public IPs.



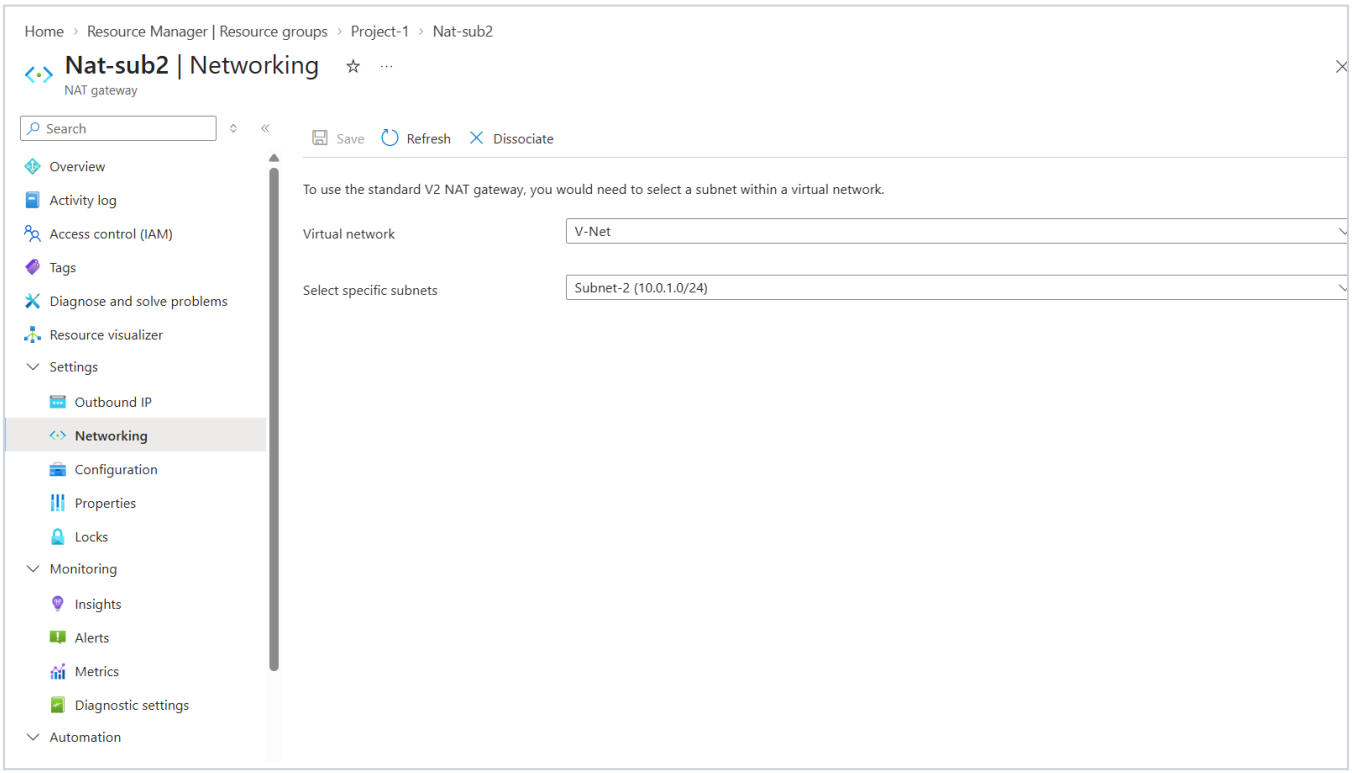
Nat-sub2 — Review + create

3.2 Associate the Gateway with Subnet-2

Virtual network	V-Net
Associated subnet	Subnet-2 (10.0.1.0/24)
Outbound IP (final)	nat-pip

The NAT gateway is bound to **Subnet-2** — the same subnet hosting the scale-set instances — so every outbound connection those VMs make is translated through the gateway's public IP, **nat-pip** (added after initial creation; see the final resource inventory in Phase 6).

Inbound internet access to the instances is still only possible through the Load Balancer (Phase 4) — the NAT gateway is strictly an outbound-only path.



Nat-sub2 | Networking — associated virtual network and subnet

6. Phase 4 — Load Balancing

PHASE 4

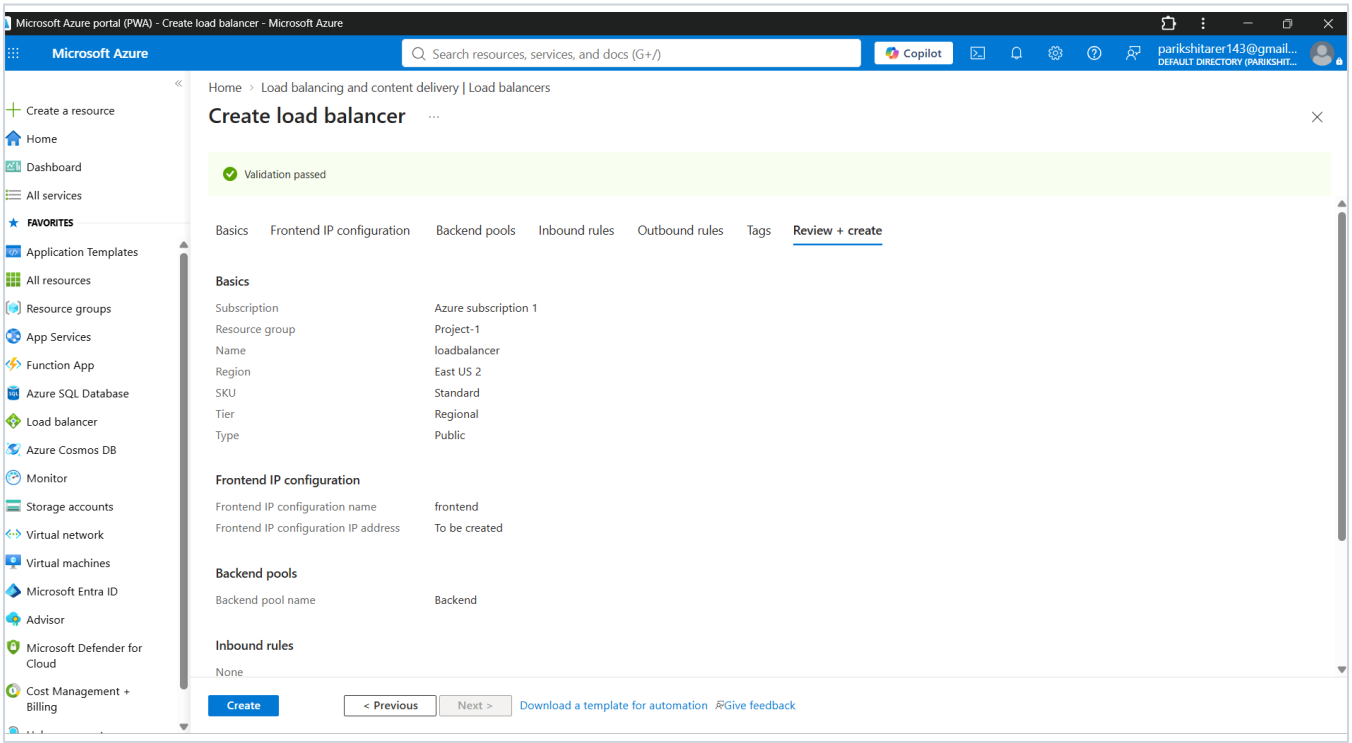
Load Balancing

Put a single, public entry point in front of the scale set and confirm it is routing traffic to healthy backend instances only.

4.1 Create the Load Balancer

Name	loadbalancer
SKU	Standard
Tier	Regional
Type	Public
Frontend IP config name	frontend
Backend pool name	Backend

A **Standard, Regional, Public** Load Balancer is created to front the scale set. Standard SKU is chosen specifically because it supports zone-redundant configurations and integrates natively with VM Scale Sets spread across availability zones.

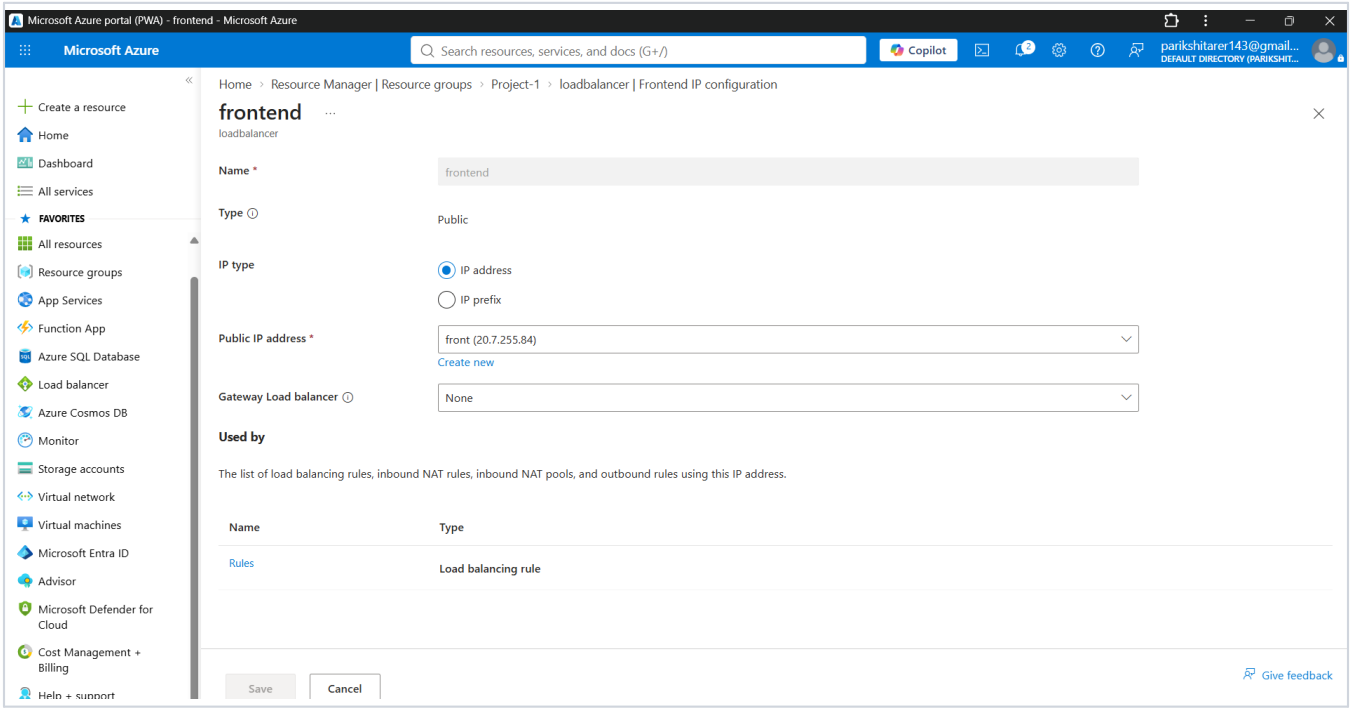


loadbalancer — Review + create

4.2 Review the Frontend IP Configuration

Frontend name	frontend
Public IP	front (20.7.255.84)
Gateway Load Balancer	None
Used by	Rules — Load balancing rule

The frontend binds a dedicated public IP address, **front (20.7.255.84)**, to the load balancer. This is the single address end users hit — decoupling the application's public identity from however many backend instances happen to be running at any moment.



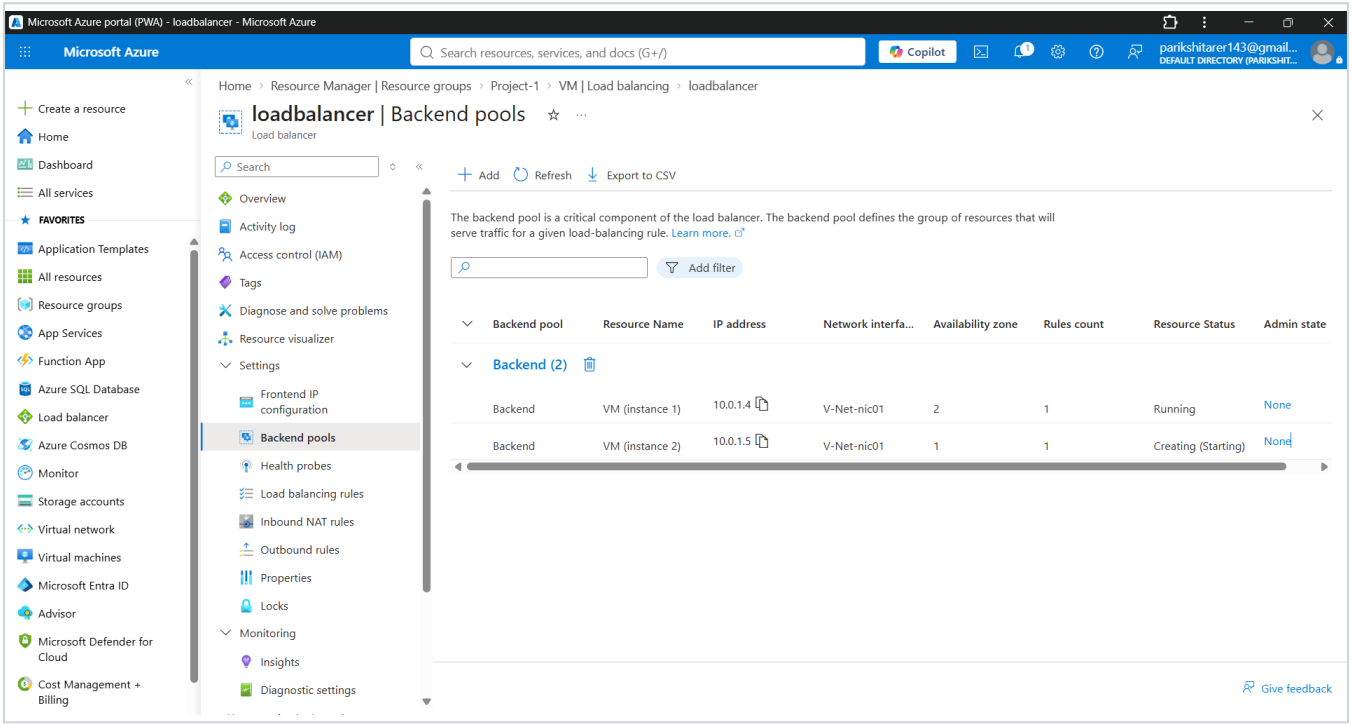
loadbalancer | Frontend IP configuration — “frontend”

4.3 Confirm the Backend Pool

Backend pool	Backend
Instance 1	10.0.1.4 — zone 2 — Running
Instance 2	10.0.1.5 — zone 1 — Creating (Starting)
Network interface	V-Net-nic01

Both scale-set instances are registered in the **Backend** pool, each in a different availability zone (zone 2 and zone 1) — so the loss of an entire physical datacenter would still leave one instance able to serve traffic.

One instance is captured mid-start here, illustrating the scale set naturally cycling members in and out as autoscale (Phase 5.3) reacts to load.

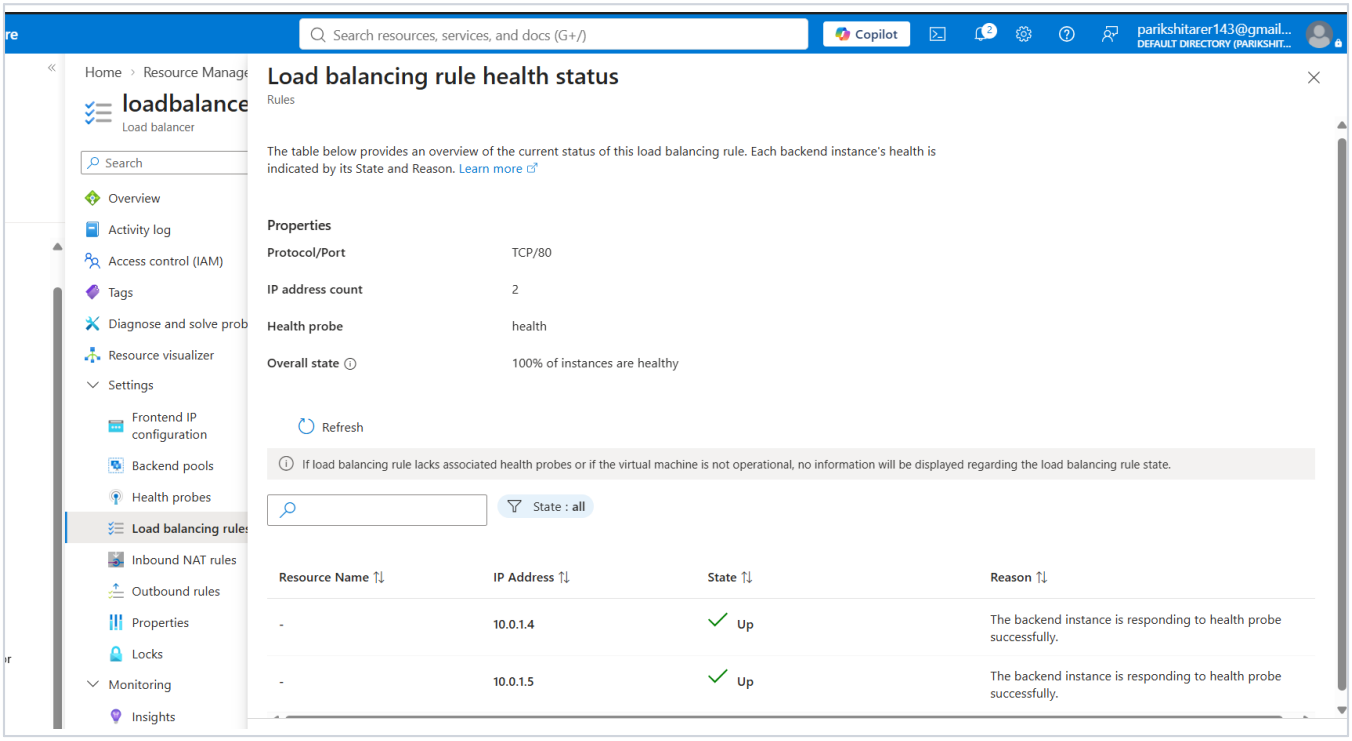


loadbalancer | Backend pools — both VMSS instances registered

4.4 Confirm Load-Balancing Rule Health

Protocol / Port	TCP / 80
Health probe	health
IP address count	2
Overall state	100% of instances are healthy
10.0.1.4	Up
10.0.1.5	Up

The health-probe status confirms both backend instances are passing their TCP/80 health check and are actively eligible to receive traffic — proof that the load-balancing rule, the NSGs, and nginx on each VM are all correctly aligned end-to-end.



Load balancing rule health status — 100% of instances healthy

7. Phase 5 — Validation, Scaling & End-to-End Test

PHASE 5

Validation, Scaling & End-to-End Test

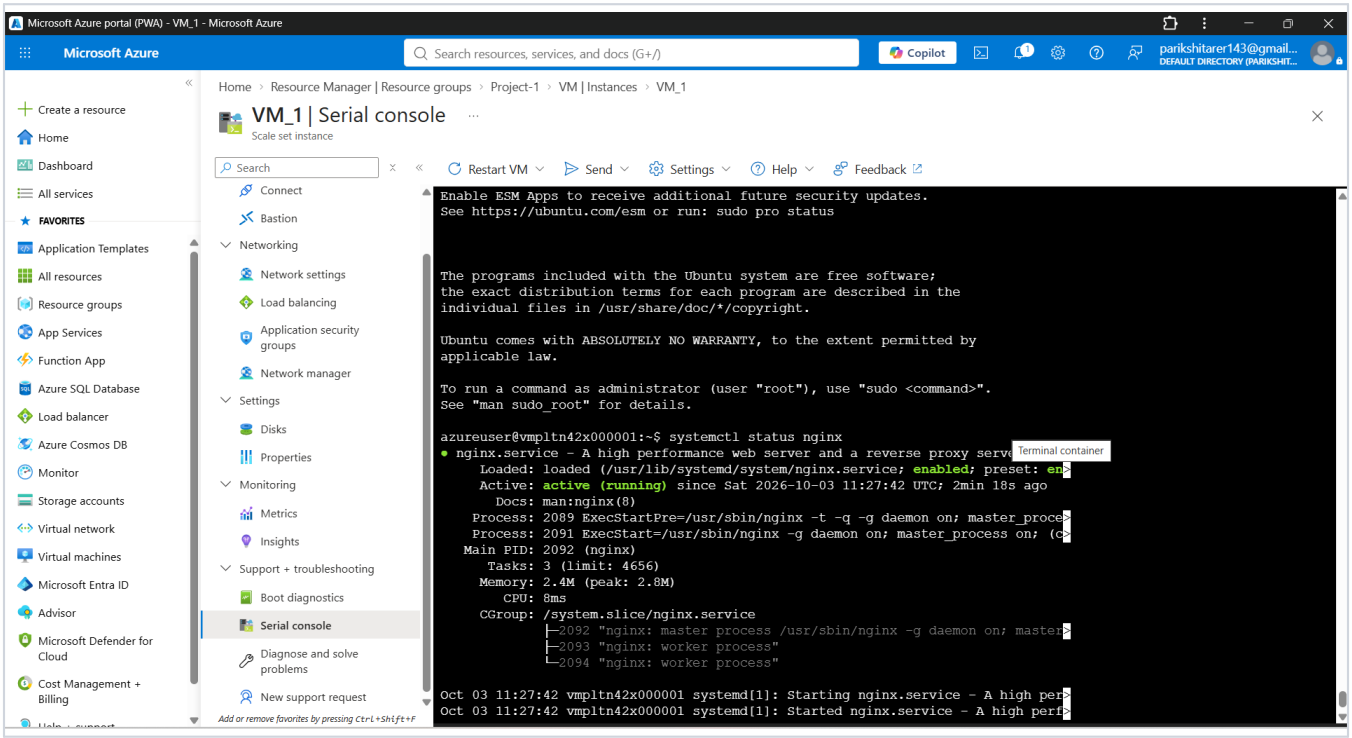
Re-confirm instance health after the load balancer attach, turn on autoscale, and prove the whole chain works from a real browser.

5.1 Re-verify the Web Server — Second Check

Instance	VM_1
State	active (running)
Started	2026-10-03 11:27:42 UTC (reset from the 09:00:58 UTC start seen in Step 2.3)

A second serial-console check later in the session shows nginx still **active** — but with a new start timestamp, meaning the instance (or at least the nginx service) restarted sometime between the two checks.

That window lines up with the Custom Script Extension update observed in Step 2.4, consistent with Azure re-applying the extension and the service coming back up cleanly afterward — exactly the kind of drift a second verification pass is meant to catch.

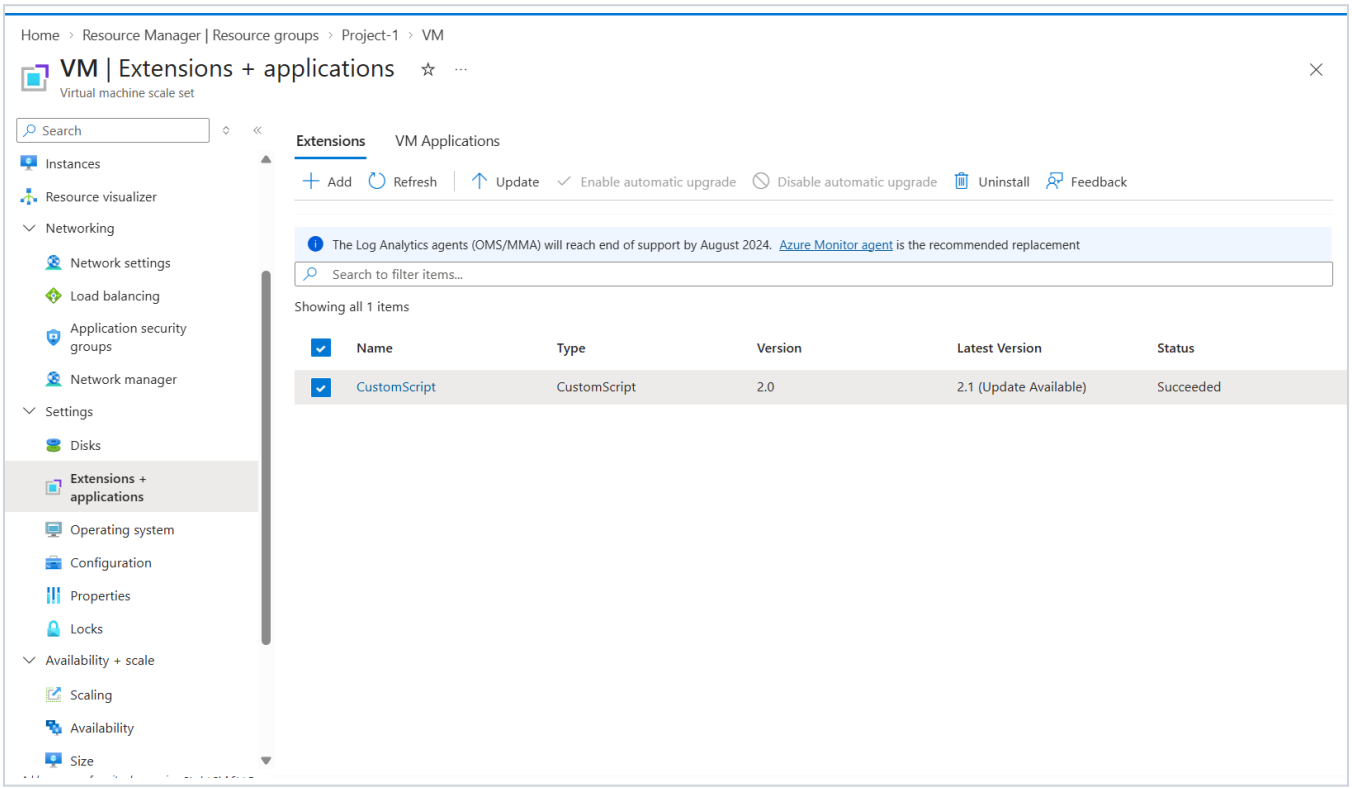


VM_1 | Serial console — systemctl status nginx, later in the session

5.2 Confirm the Extension Finished

Extension	CustomScript
Status	Succeeded

The Extensions + applications blade now reports **Succeeded** rather than **Updating**, confirming the rollout observed in Step 2.4 completed cleanly with no failed instances.



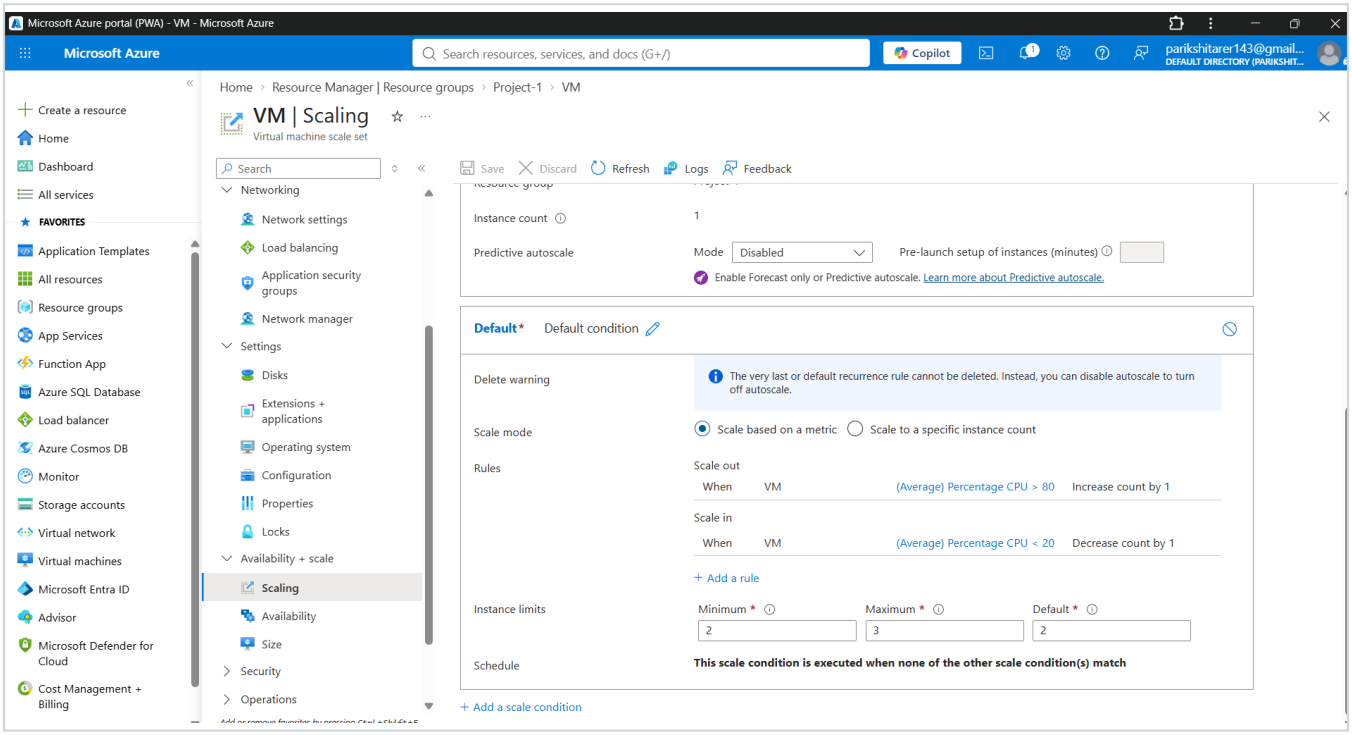
VM | Extensions + applications — CustomScript now Succeeded

5.3 Configure Autoscale Rules

Scale mode	Scale based on a metric
Scale out	Average Percentage CPU > 80 → increase count by 1
Scale in	Average Percentage CPU < 20 → decrease count by 1
Instance limits	Minimum 2 — Maximum 3 — Default 2

Autoscale is configured directly on the scale set (the **VMautoscale** resource created back in Step 2.1) so capacity follows demand automatically: the fleet grows by one instance whenever average CPU crosses **80%**, and shrinks by one whenever it drops below **20%**.

The fleet always stays within a floor of **2** and a ceiling of **3** instances — enough headroom to absorb a traffic spike without runaway cost.



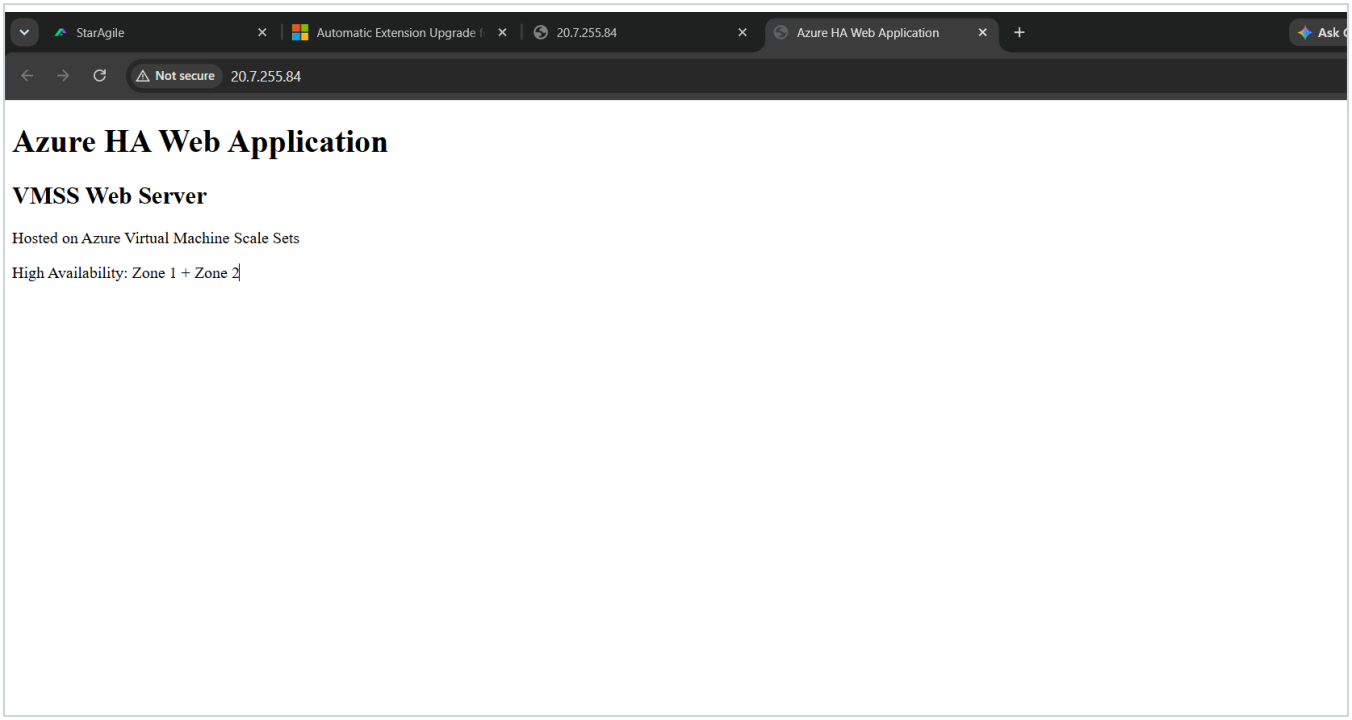
VM | Scaling — metric-based autoscale rules

5.4 End-to-End Browser Test

URL tested	http://20.7.255.84 — the Load Balancer's public frontend IP
Result	“Azure HA Web Application” page renders correctly

Hitting the load balancer's public IP directly in a browser returns the custom nginx page authored in Step 2.2 — the clearest possible proof that the full chain (Internet → Public IP → Load Balancer → Backend Pool → NSGs → nginx) works end-to-end.

The browser's “Not secure” badge is expected: the NSG opened port 443 for future use, but no TLS certificate or HTTPS listener has been configured on the load balancer yet (see Recommendations).

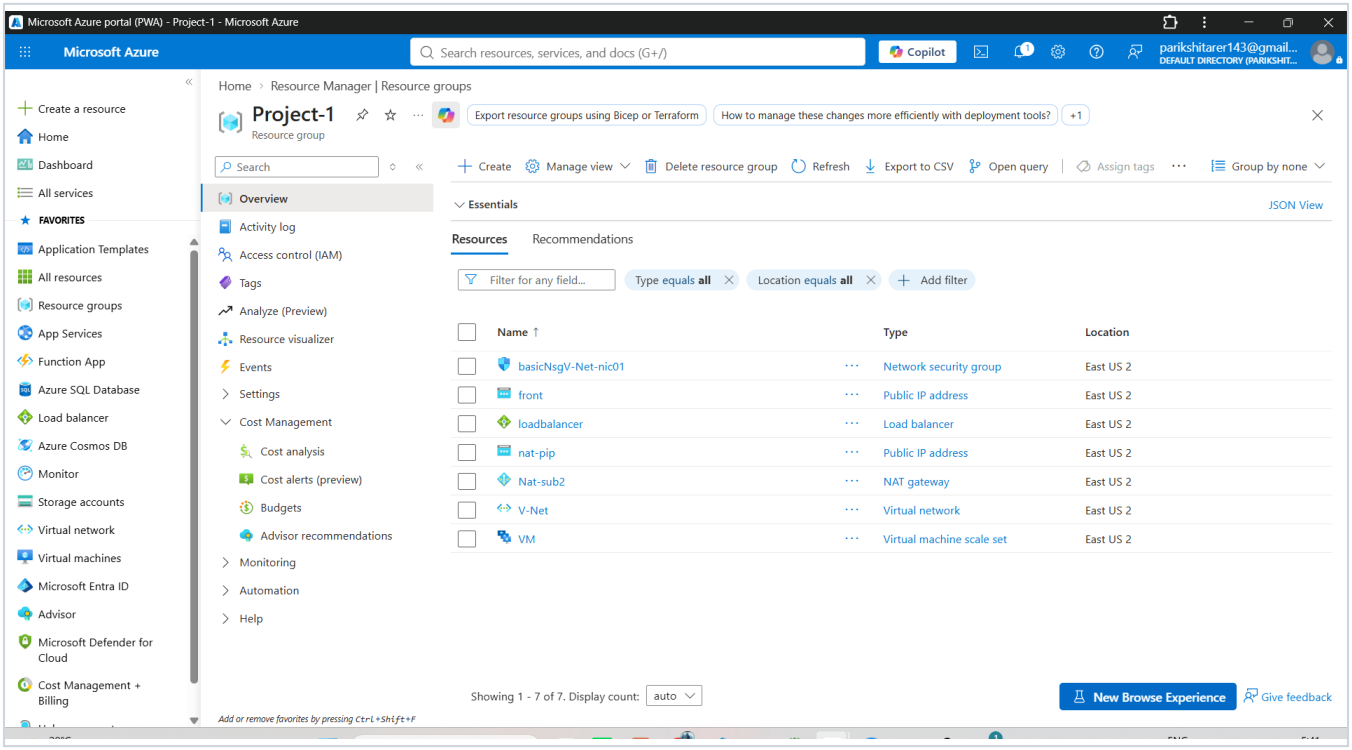


Browser request to the Load Balancer's public IP — page renders correctly

8. Final Review & Resource Inventory

8.1 Resource Group Inventory

A closing snapshot of every resource that exists in **Project-1** once the build is complete — each one tracing back to a phase documented above. See the Resource Inventory table for a role-by-role breakdown.



Project-1 — final resource group contents (7 resources)

Resource Inventory

Resource Name	Type	Location	Role in Architecture
Project-1	Resource group	East US 2	Container for every resource below
V-Net	Virtual network	East US 2	10.0.0.0/16 address space, 3 subnets
NSW-Subnet-1	Network security group	East US 2	Subnet-level inbound allow-list (80, 443)
basicNsgV-Net-nic01	Network security group	East US 2	NIC-level inbound allow-list (auto-created with VMSS)
VM	Virtual machine scale set	East US 2	Ubuntu 24.04 LTS web tier, 2–3 instances
Nat-sub2	NAT gateway	East US 2	Outbound internet for Subnet-2
nat-pip	Public IP address	East US 2	Outbound IP used by Nat-sub2
loadbalancer	Load balancer	East US 2	Standard, Public — distributes inbound traffic
front	Public IP address	East US 2	Frontend IP for loadbalancer (20.7.255.84)

9. Recommendations & Next Steps

Observations from the build above that would strengthen the architecture if this project continues beyond the initial deployment.

Enable HTTPS on the Load Balancer

The NSG already allows inbound TCP 443, but no TLS certificate or HTTPS load-balancing rule has been configured yet — the browser test in Phase 5.4 correctly shows “Not secure.” Add a certificate (e.g., via Azure Key Vault) and a matching load-balancing rule, or front the solution with Azure Application Gateway / Front Door for managed TLS.

Tag every resource

Both the Resource Group and the Virtual Network were created with Tags left as “None.” Adding consistent tags (Environment, Owner, CostCenter) would make cost tracking and ownership much easier as the project grows.

Reconcile the two NSGs

Traffic to each VMSS instance is filtered by both NSW-Subnet-1 (subnet-level) and basicNsgV-Net-nic01 (NIC-level, auto-generated at VMSS creation). Document both rule sets together so a future change to one doesn't unexpectedly block traffic the other was meant to allow.

Add monitoring and alerting

No diagnostic settings or alert rules were configured during this build. Wiring the Load Balancer health probe and the VMSS autoscale events into Azure Monitor alerts would turn the manual serial-console checks performed in Phases 2.3 and 5.1 into automatic, proactive notifications.

Reserve Subnet-1 and Subnet-3 deliberately

Subnet-1 (10.0.0.0/24) and Subnet-3 (10.0.2.0/24) were provisioned but are not yet used by any resource. Documenting their intended purpose now (e.g., a future data tier or a dedicated gateway subnet) will keep the address plan coherent as the architecture expands.

10. Conclusion

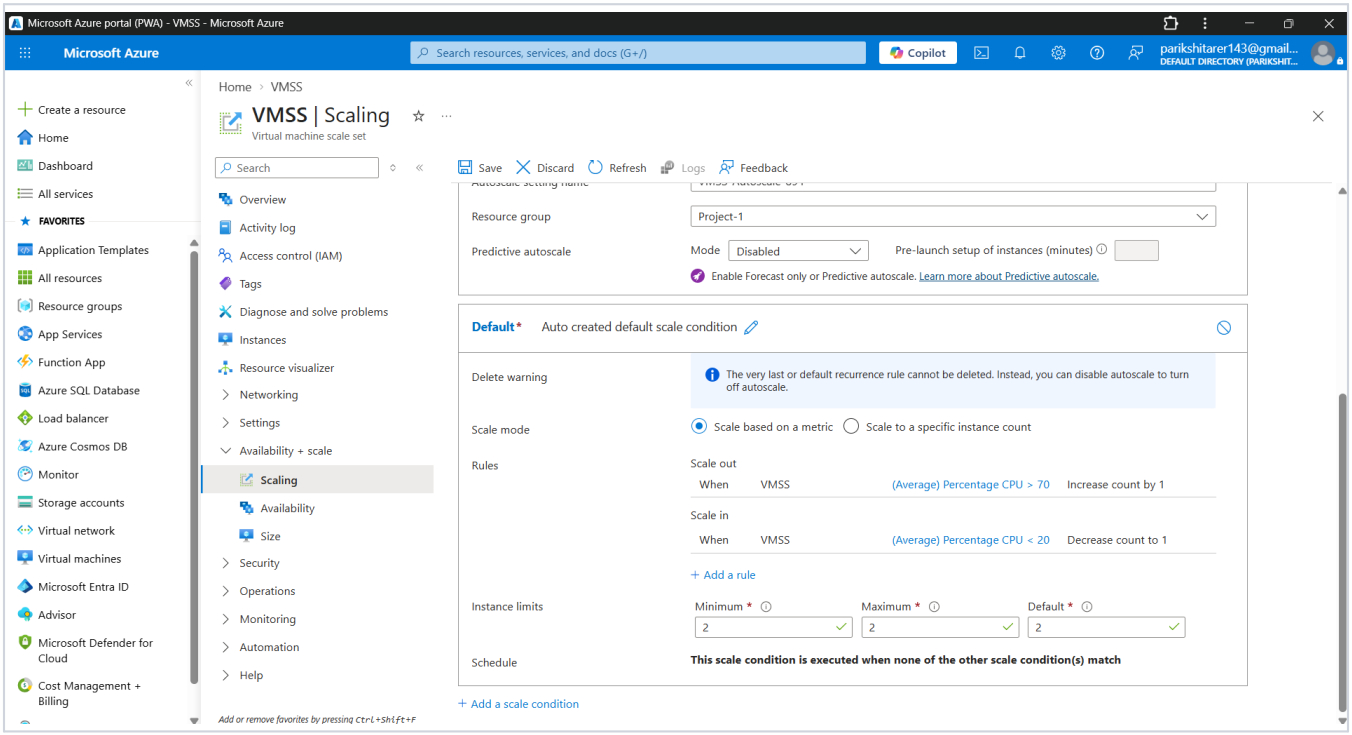
By the end of the build, Project-1 contained a complete, self-healing web tier: a 2-zone, auto-scaling Virtual Machine Scale Set running nginx — installed and configured entirely through a Custom Script Extension rather than manual SSH — sitting behind a Standard public Load Balancer whose health probe reported both backend instances as 100% healthy. Outbound connectivity for the private-subnet instances was handled centrally through a NAT Gateway, and inbound access was restricted to HTTP/HTTPS by a two-layer NSG allow-list. The architecture was validated from two independent angles — the Azure Serial Console confirming nginx's service state directly on the instance, and a live browser request against the Load Balancer's public IP confirming the entire request path end-to-end — giving confidence that the deployment is both highly available and correctly wired together.

Appendix A — Supplementary Autoscale Exercise (October 1)

Scale out	Average Percentage CPU > 70 → increase count by 1
Scale in	Average Percentage CPU < 20 → decrease count to 1
Instance limits	Minimum 2 — Maximum 2 — Default 2

Prior to the October 3 build documented above, a separate, standalone scale set named **VMSS** was used to trial autoscale settings — a different CPU threshold (70% rather than 80%) and a scale-in rule that drops straight to a floor of 1 instance.

This screenshot is included for completeness and traceability, but **VMSS** is not part of the Project-1 resource group documented in Phases 1–6; it predates that build by two days and uses different scaling thresholds.



A standalone scale set, “VMSS” — an earlier, separate autoscale trial